

Workflows

AUTHOR

Erola Fenollosa

Workflow in practice: Pipes and functions

The gapminder dataset

The **Gapminder** dataset contains longitudinal data describing social and economic indicators for countries around the world from 1952 to 2007, measured at five-year intervals. For each country and year, it includes life expectancy at birth (`lifeExp`), population size (`pop`), and gross domestic product per capita (`gdpPercap`), along with the country name and its continent. The structure is tidy: each row represents one country in one specific year, making it ideal for analysing changes over time, comparing continents, and building reproducible workflows using grouping, filtering, summarising, and visualisation in R.

```
#install.packages("gapminder")
library(gapminder)
```

Warning: package 'gapminder' was built under R version 4.5.2

```
data(gapminder)

library(dplyr) # piping and data wrangling
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(ggplot2) # Plotting
```

Use some functions you know to take a look at the data (e.g. `str`, `head`, `names`, `summary`, `dim`...).

```
# Write your code here:
# ANSWER
```

Below is a curated set of class exercises (increasing difficulty). Each item includes the task, the *type* (write / pipe-decode / debug / predict / design).

FUNCTION CHALLENGE LADDER

Level 0. Warm-up. Pipes decoding (trace the pipeline)

(Decode) Explain this pipeline in plain English (or pseudocode). Write step-by-step what happens to the data at each pipe stage and what the final table contains.

```
gapminder %>% filter(year == 2007)
```

A tibble: 142 × 6

	country	continent	year	lifeExp	pop	gdpPercap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
1	Afghanistan	Asia	2007	43.8	31889923	975.
2	Albania	Europe	2007	76.4	3600523	5937.
3	Algeria	Africa	2007	72.3	33333216	6223.
4	Angola	Africa	2007	42.7	12420476	4797.
5	Argentina	Americas	2007	75.3	40301927	12779.
6	Australia	Oceania	2007	81.2	20434176	34435.
7	Austria	Europe	2007	79.8	8199783	36126.
8	Bahrain	Asia	2007	75.6	708573	29796.
9	Bangladesh	Asia	2007	64.1	150448339	1391.
10	Belgium	Europe	2007	79.4	10392226	33693.

i 132 more rows

ANSWER:

```
gapminder %>% group_by(continent)
```

A tibble: 1,704 × 6

Groups: continent [5]

	country	continent	year	lifeExp	pop	gdpPercap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
1	Afghanistan	Asia	1952	28.8	8425333	779.
2	Afghanistan	Asia	1957	30.3	9240934	821.
3	Afghanistan	Asia	1962	32.0	10267083	853.
4	Afghanistan	Asia	1967	34.0	11537966	836.
5	Afghanistan	Asia	1972	36.1	13079460	740.
6	Afghanistan	Asia	1977	38.4	14880372	786.
7	Afghanistan	Asia	1982	39.9	12881816	978.
8	Afghanistan	Asia	1987	40.8	13867957	852.
9	Afghanistan	Asia	1992	41.7	16317921	649.
10	Afghanistan	Asia	1997	41.8	22227415	635.

i 1,694 more rows

ANSWER:

```
gapminder %>%  
  filter(year == 2007) %>%  
  mutate(lifeExp_plus5 = lifeExp + 5)
```

```
# A tibble: 142 × 7
  country      continent  year lifeExp      pop gdpPercap lifeExp_plus5
  <fct>      <fct>    <int> <dbl>    <int>    <dbl>      <dbl>
1 Afghanistan Asia      2007  43.8  31889923    975.      48.8
2 Albania    Europe    2007  76.4   3600523   5937.      81.4
3 Algeria    Africa    2007  72.3  33333216   6223.      77.3
4 Angola     Africa    2007  42.7  12420476   4797.      47.7
5 Argentina  Americas  2007  75.3  40301927  12779.     80.3
6 Australia  Oceania   2007  81.2  20434176  34435.     86.2
7 Austria    Europe    2007  79.8   8199783   36126.     84.8
8 Bahrain    Asia      2007  75.6    708573   29796.     80.6
9 Bangladesh Asia      2007  64.1 150448339   1391.     69.1
10 Belgium   Europe    2007  79.4  10392226   33693.     84.4
# i 132 more rows
```

ANSWER:

```
gapminder %>% group_by(continent) %>% summarise(n = n())
```

```
# A tibble: 5 × 2
  continent      n
  <fct>    <int>
1 Africa    624
2 Americas  300
3 Asia      396
4 Europe    360
5 Oceania    24
```

ANSWER:

```
gapminder %>%
  group_by(country) %>%
  summarise(max_lifeExp = max(lifeExp, na.rm = TRUE))
```

```
# A tibble: 142 × 2
  country      max_lifeExp
  <fct>      <dbl>
1 Afghanistan  43.8
2 Albania      76.4
3 Algeria      72.3
4 Angola       42.7
5 Argentina    75.3
6 Australia    81.2
7 Austria      79.8
8 Bahrain      75.6
9 Bangladesh   64.1
10 Belgium     79.4
# i 132 more rows
```

```
gapminder %>%
  group_by(country) %>%
  summarise(max_lifeExp = max(lifeExp, na.rm = TRUE)) %>%
```

```
arrange(desc(max_lifeExp)) %>%
slice_head(n = 3)
```

```
# A tibble: 3 × 2
  country      max_lifeExp
  <fct>        <dbl>
1 Japan          82.6
2 Hong Kong, China 82.2
3 Iceland        81.8
```

ANSWER:

What is the difference between these codes. Why do the first and the last produce (apparently) the same result?

```
gapminder %>%
  group_by(country) %>%
  summarise(max_lifeExp = max(lifeExp, na.rm = TRUE)) %>%
  arrange(desc(max_lifeExp)) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  country      max_lifeExp
  <fct>        <dbl>
1 Japan          82.6
2 Hong Kong, China 82.2
3 Iceland        81.8
4 Switzerland    81.7
5 Australia      81.2
```

```
gapminder %>%
  arrange(desc(lifeExp)) %>%
  slice_head(n = 5) %>%
  group_by(country) %>%
  summarise(max_lifeExp = max(lifeExp, na.rm = TRUE))
```

```
# A tibble: 4 × 2
  country      max_lifeExp
  <fct>        <dbl>
1 Hong Kong, China 82.2
2 Iceland        81.8
3 Japan          82.6
4 Switzerland    81.7
```

```
gapminder %>%
  arrange(desc(lifeExp)) %>%
  distinct(country, .keep_all = TRUE) %>%
  select(country, lifeExp) %>%
  rename(max_lifeExp = lifeExp) %>%
  arrange(desc(max_lifeExp)) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  country      max_lifeExp
  <fct>         <dbl>
1 Japan          82.6
2 Hong Kong, China 82.2
3 Iceland         81.8
4 Switzerland     81.7
5 Australia       81.2
```

ANSWER:

Consider the following code (delete the "#"), and explain why the filter does not work as intended. What dataset is being filtered at that stage? Correct the pipeline so that you count rows per continent **only for the year 2007**.

```
# gapminder %>%
#   group_by(continent) %>%
#   summarise(n = n()) %>%
#   filter(year == 2007)
```

ANSWER:

(Rewrite) Convert to piped form:

```
# Step 1: keep only rows for Europe
step1 <- filter(gapminder, continent == "Europe")

# Step 2: add derived columns (total GDP and GDP in millions)
step2 <- mutate(step1,
  gdp_total = gdpPercap * pop,
  gdp_per_million = gdp_total / 1e6)

# Step 3: create grouping by country
step3 <- group_by(step2, country)

# Step 4: summarise to get median GDP per million per country
step4 <- summarise(step3,
  median_gdp_million = median(gdp_per_million, na.rm = TRUE))

# Step 5: arrange the result from smallest -> largest median (use desc() if you want largest first)
step5 <- arrange(step4, median_gdp_million)

# Show the final result
step5
```

```
# A tibble: 30 × 2
  country      median_gdp_million
  <fct>         <dbl>
1 Montenegro    4416.
2 Iceland       4902.
3 Albania       8586.
4 Bosnia and Herzegovina 12670.
5 Slovenia      27553.
```

6 Ireland	40198.
7 Croatia	41307.
8 Slovak Republic	51551.
9 Bulgaria	55576.
10 Finland	81705.

i 20 more rows

ANSWER:

Level 1. The Pain (No Functions Yet)

Which continent had the higher life expectancy in 2007?

ANSWER

And 1997?

ANSWER

Obtain a list of all the countries found in the gapminder dataset, consider using pipes, but feel free to use other ways.

ANSWER

Check the life expectancy in “United Kingdom” for all possible years (you might need to copy-paste similar lines multiple times). Make sure you know how the UK is named in the dataset by using the previous call.

ANSWER

What part of this code varies? What part repeats?

ANSWER:

Level 2. Write a function

The function needs to take a year as argument, and return a tibble with the mean life expectancy for all continents on the explicated year (year_value). This might be your first function, so let’s take it easy. First explain which tasks will you be doing. It is easier than it seems. You just have to copy what you designed previously but using the “year_value” instead of the absolute year. Note that the function definition will not return anything, but the following chunk will return the result.

```
mean_life_by_continent <- function(year_value) {  
  # ADD TASKS HERE  
}
```

Use the function. Change the years to check it works:

```
mean_life_by_continent(2007)
```

NULL

```
mean_life_by_continent(1962)
```

NULL

Level 3. Improve the design

The function takes the dataset internally, and therefore is not fully reusable. Modify the function so it takes a second argument, which is the dataframe(df). Make sure you use both "df" and "year_value" inside the functions.

```
mean_by_continent_year <- function(df, year_value) {  
  # ADD TASKS HERE  
}
```

```
mean_by_continent_year(gapminder, 2007)
```

NULL

Level 4. Generalise to other variables

What if we want GDP per capita instead of life expectancy? Modify the function so it can take any variable. Some parts of the code have been already written. If you are not sure what specific parts of the code are doing, feel free to explore online and add your own explanation.

```
mean_by_continent_year_var <- function(df, year_value, var_name) {  
  df %>%  
    filter #ADD CODE  
    group_by # ADD CODE  
    summarise(mean_value = mean(.data[[var_name]], na.rm = TRUE)) %>%  
    arrange # ADD CODE  
}
```

```
# mean_by_continent_year_var(gapminder, 2007, "lifeExp")  
# mean_by_continent_year_var(gapminder, 2007, "gdpPercap")
```

Level 5. Add ranking control

Modify your function to return only the top N continents. Add an argument called top_n = 3.

Level 6. Debugging

How would you use this function? What is wrong with it?

```
bad_function <- function(df, year_value, var_name) {  
  gapminder %>%  
    group_by(continent) %>%  
    summarise(mean_value = mean(var_name))  
}
```

Level 7. Test yourself. Are you able to explain this function?

```
# plot_trend: plot time trend of the mean of a variable for a continent  
#  
# Arguments:  
#   df           - data frame containing at least columns "year" and "continent" and the variable name  
#   continent_name- character scalar, the continent to filter on (e.g. "Europe")  
#   var_name      - character scalar, name of the numeric variable to summarise (e.g. "lifeExp" or "gdpPercap")  
#   smooth        - logical (default TRUE). If TRUE the plot includes a smooth trend line (geom_smooth)  
#  
# Behaviour:  
#   - computes the mean of var_name for each year within the chosen continent (NA values ignored)  
#   - builds and returns a ggplot object showing year (x) vs mean value (y) with points and a connecting line  
#   - if smooth == TRUE, adds a smooth trend layer  
#   - does NOT modify the input df and does NOT save any files or objects to disk  
#  
# Requirements:  
#   - dplyr and ggplot2 should be available (library(dplyr); library(ggplot2))  
#   - var_name must exist in df; a clear error is thrown otherwise  
#  
plot_trend <- function(df, continent_name, var_name, smooth = TRUE) {  
  # defensive checks  
  if (!is.data.frame(df)) stop("df must be a data frame")  
  if (!"year" %in% names(df)) stop("df must contain a 'year' column")  
  if (!"continent" %in% names(df)) stop("df must contain a 'continent' column")  
  if (!is.character(continent_name) || length(continent_name) != 1) stop("continent_name must be a single string")  
  if (!is.character(var_name) || length(var_name) != 1) stop("var_name must be a single string")  
  if (!var_name %in% names(df)) stop(paste0("Variable '", var_name, "' not found in df"))  
  if (!is.logical(smooth) || length(smooth) != 1) stop("smooth must be a single logical value (TRUE or FALSE)")  
  
  # create a small summary table (this is local to the function; we do not assign to global)  
  summary_tbl <- df %>%  
    dplyr::filter(continent == continent_name) %>%           # keep only rows for the requested continent  
    dplyr::group_by(year) %>%                                # group by year so summaries are per-year  
    dplyr::summarise(  
      mean_value = mean(.data[[var_name]], na.rm = TRUE),    # compute mean of chosen variable, ignoring NA values  
      .groups = "drop"                                       # drop grouping after summarise (return ungrouped tibble)  
    )  
  
  # build the ggplot object
```

```

p <- ggplot2::ggplot(summary_tbl, ggplot2::aes(x = year, y = mean_value)) +
  ggplot2::geom_point() +      # show the yearly mean as points
  ggplot2::geom_line() +      # connect the points with lines to visualise the trend
  ggplot2::labs(
    x = "Year",
    y = paste0("Mean ", var_name),
    title = paste0("Trend of mean ", var_name, " in ", continent_name)
  ) +
  ggplot2::theme_minimal()     # a clean, minimal theme for readability

# optionally add a smooth trend (loess by default; automatic choice by ggplot2)
if (isTRUE(smooth)) {
  p <- p + ggplot2::geom_smooth(se = TRUE, method = "loess")
}

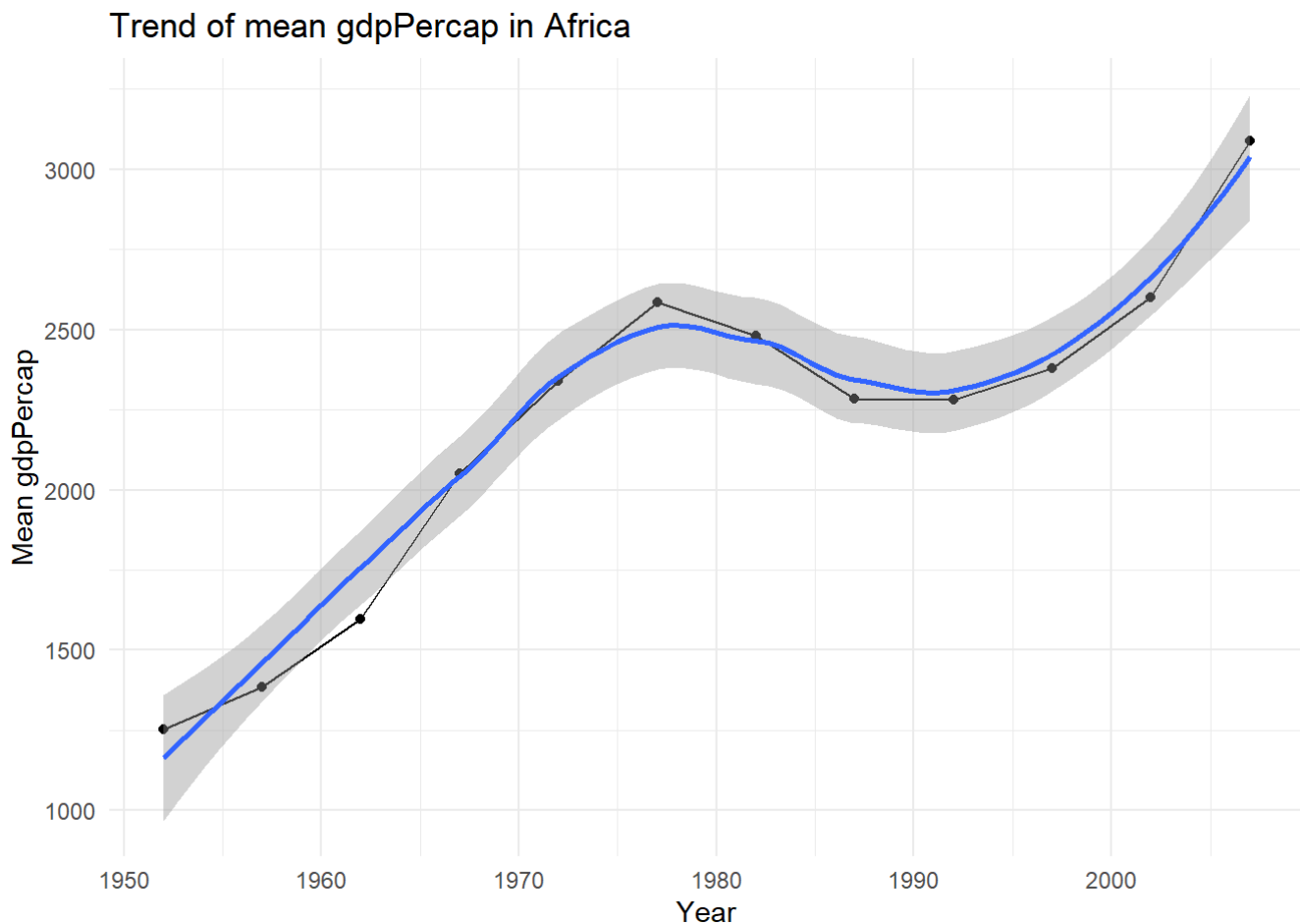
# Return the ggplot object. In an interactive session printing the object will display the plot.
p
}

```

Change the countries or variables.

```
plot_trend(gapminder, "Africa", "gdpPercap", smooth = TRUE)
```

`geom\_smooth()` using formula = 'y ~ x'



FINAL CHALLENGE

Would you be able to modify the created function to change the color of the plot for example?

How would you use the function to quickly obtain a plot for all the continents or/and countries? Could you code it? Start first with designing the pseudocode. How much time do you feel this will save?

ANSWER